Task Description:

Privacy-Preserving Source Code Vulnerability Detection and Repair using Retrieval-Augmented LLMs for Visual Studio Code

Md Hafizur Rahman

Traditional static application security testing (SAST) tools, such as Semgrep and CodeQL, are effective for detecting predefined vulnerability patterns but cannot reason about cross-file or context-dependent weaknesses. Cloud-based language model assistants provide stronger semantic analysis yet compromise data confidentiality and reproducibility because proprietary source code must leave the local environment. This thesis aims to design and evaluate a fully local, privacy-preserving secure coding assistant for Visual Studio Code that integrates large language models (LLMs) with Retrieval-Augmented Generation (RAG). All processing occurs on the developer's machine under a strict zero-egress policy to ensure that source code and intermediate data remain confidential.

The proposed assistant combines two complementary modes: real-time inline diagnostics that deliver immediate vulnerability alerts, and asynchronous audit and question–answering modes that use offline retrieval of vulnerability information such as CWE, CVE, and OWASP entries. A modular architecture separates the LLM inference layer, the local retrieval pipeline, and the Visual Studio Code extension interface. The system enforces privacy through local embeddings, signed corpora, and network isolation, while reproducibility is achieved through deterministic decoding, version-pinned dependencies, and containerized execution. The threat model assumes local adversaries capable of attempting code exfiltration, retrieval poisoning, or prompt injection, mitigated through corpus signing, network isolation, and provenance verification.

Evaluation will be conducted on benchmark and real-world JavaScript/TypeScript code. The comparison will be conducted across benchmark datasets (Juliet and OWASP Benchmark, approximately 40–50 test cases mapped to CWE identifiers) and one actively maintained real-world JavaScript/TypeScript project with documented vulnerabilities and verified patches. If the optional SAST baseline is executed, Semgrep and a scoped CodeQL subset will be included for contextual comparison. Evaluation metrics include precision, recall, F1-score, latency, resource usage, privacy validation, and reproducibility, all measured under deterministic local execution.